

CSE 151B/251B Final Report

Math Reasoning Inference with vLLM, Structured Prompting, and Deterministic Answer Repair

Keith Gong
CSE 151B/251B Kaggle Competition
University of California, San Diego
x4gong@ucsd.edu

June 1, 2026

Abstract

This project studies automatic solution generation for mixed multiple-choice and free-response mathematics problems. The final system uses the base model `Qwen/Qwen3-4B-Thinking-2507`, served with vLLM and bitsandbytes quantization on a single NVIDIA A40 GPU. The final solution is not a fine-tuned model; it is an inference system. It combines three practical components: preserving the strong baseline multiple-choice pipeline, using a concise free-response prompt that controls answer count and numerical precision, and applying a deterministic answer-repair layer that normalizes or replaces malformed final answers when the model’s reasoning trace contains a recoverable answer. On the public free-response set, the repair pipeline improved full-set free-response accuracy from 430/751 (57.26%) to 544/751 (72.44%). On the Kaggle public leaderboard, the final submitted CSV reached 0.674 after staged improvements from 0.650, 0.653, 0.667, and 0.671. The private leaderboard score was not available at report-writing time. The final repository is https://github.com/xibahbah/151B_SP26_Competition, and the required single entry point is `run_inference()`.

1 Introduction

Problem Definition. The task is to build an inference system for a Kaggle-style math reasoning competition. Each input item is a natural-language math problem. Some items are multiple-choice questions (MCQ), where the desired output is a single answer letter, while others are free-response questions (FRQ), where the output may contain one or more numeric, symbolic, or categorical answers corresponding to `[ANS]` blanks. The required submission is a CSV file with columns `id`, `response`. The `response` field must contain the model’s full generated text, including its final boxed answer; the evaluator then extracts and judges the final answer.

This is not a standard supervised classification task with a small fixed label set. The input space includes algebra, statistics, probability, geometry, trigonometry, finance, exponential models, word problems, and multi-part answer formats. The output space is heterogeneous: a correct answer can be a letter, integer, decimal, interval, expression, ordered list, or a mixture of these. Therefore, the project is best viewed as controlled mathematical text generation followed by answer extraction. Given a problem x_i , the system generates a response r_i using a language model and then optionally transforms that response through deterministic post-processing $g(r_i, x_i)$ before writing it to the CSV.

The public dataset contains labels and was used for debugging and measuring progress. The private dataset has the same submission interface but no labels. This split shaped the project: public results could be used to understand error types, but the final submitted CSV still had to be produced from private problem text only. The main engineering goal was therefore to make a private-set inference function that behaves consistently with the public-set validation pipeline.

Problem Significance. Reliable mathematical reasoning systems are useful for tutoring, automated grading, scientific computing assistance, and decision support. A student-facing system must not only reason correctly, but also present answers in the format that a grader or downstream tool expects. This competition emphasizes that second requirement. Many model outputs contained the right calculation in the body of the response but failed because the final boxed answer was missing, had the wrong answer count, included units, used commas as thousands separators, or rounded inconsistently.

The project therefore targets a realistic bottleneck for deploying large language models (LLMs): correctness is jointly determined by reasoning and interface discipline. A model that computes a value but wraps the wrong substring in `\boxed{}` still receives zero credit. In real applications, these failures look like malformed API outputs, brittle JSON, or ambiguous spreadsheet formulas. Improving answer extraction and final formatting is a practical contribution, not merely a cosmetic one.

Technical Challenge. The project was difficult for five reasons. First, the dataset mixes MCQ and FRQ examples, and changes that improve one type can hurt the other. In early experiments, modifying MCQ prompting and decoding reduced MCQ accuracy, so the final system preserves the original MCQ path. Second, the FRQ answer space is broad: some questions require exact fractions, some require decimal precision, and others require multiple ordered values. Third, long-thinking models can over-generate. Increasing maximum output tokens from 4096 to 6144 allowed more reasoning, but it also increased rambling and sometimes placed the correct answer before a later contradictory answer. Fourth, the system had to run on a single A40 with finite memory and time; full private inference took approximately 10–12 hours. Fifth, evaluation was brittle: the free-response judger is sensitive to formatting, answer ordering, and extraction, so engineering work on post-processing mattered almost as much as model selection.

The most surprising technical issue was that “more reasoning” was not equivalent to more accuracy. The model often solved the problem, then continued second-guessing itself until the final boxed answer became a paragraph fragment, a contradictory value, or an unfinished sentence. In other cases, it put the right answer in the middle of the response and a wrong answer at the end. This is why the final system treats answer extraction as a separate module, not as a trivial regex.

Contributions. The final solution makes four main contributions:

- **Type-specific inference.** The pipeline separates MCQ and FRQ inference. It preserves the starter-style MCQ prompt and high-token decoding while using a stricter FRQ prompt and deterministic generation for free-response items.
- **A concise FRQ prompt with structural constraints.** The FRQ prompt explicitly counts `[ANS]` blanks, enforces one final boxed answer, bans thousands separators, and tells the model to stop after the final answer.
- **A deterministic repair layer.** The repair module extracts last boxed answers, answer markers, numeric expressions, labelled values, compact option letters, and known symbolic forms. It also contains template-specific solvers for recurring math patterns.
- **A reproducible single-entry submission pipeline.** The final repository exposes `run_inference()` and a notebook wrapper that execute the whole pipeline end-to-end: model loading, MCQ inference, FRQ inference, repair, merge, and CSV export.

2 Related Work

Large language models for mathematical reasoning. Transformers [5] enabled modern large language models by scaling attention-based sequence modeling. GPT-style models demonstrated that sufficiently large language models can solve many tasks from prompts alone [1]. The final model family in this project, Qwen3, is a modern open-weight language-model family with explicit thinking variants [8, 4]. Mathematical reasoning requires more than memorized facts: models must decompose questions, execute symbolic or numerical operations, and produce a final answer that is

extractable by an evaluator. Chain-of-thought prompting showed that asking models to write intermediate reasoning can improve multi-step performance [7]. However, this project also found a limitation of unconstrained reasoning: long answers can bury or overwrite the correct final answer. For this competition, the best-performing system did not simply ask for more reasoning; it controlled where the answer appears and how it is formatted.

Sampling, self-consistency, and serving. Self-consistency samples multiple reasoning paths and selects an answer by agreement [6]. I explored a limited version of this idea with multi-generation runs on previously missed FRQ examples. It was expensive and did not produce enough reliable gains under the time budget, so it did not become the final approach. Efficient serving mattered more. vLLM’s PagedAttention-style memory management improves throughput and allows large models to run more efficiently during inference [3]. The final system used vLLM because full-set inference with Hugging Face Transformers was too slow for repeated experimentation and because vLLM made it possible to run the same base model repeatedly on a single A40 GPU. I also tested LoRA-style fine-tuning [2], but the adapters overfit formatting or mismatched external data and were rejected.

Post-processing as a deployment tool. Many LLM papers evaluate whether a model can reason, but this competition also evaluates whether the final response is machine-readable. The repair system is related to output normalization and answer verification: it does not change the private problem statement or use private labels, but it tries to ensure that the final answer is the answer the model actually intended. The final design therefore treats post-processing as part of the model-serving stack. This is similar to production LLM systems where generated text is passed through parsers, validators, and fallback rules before being consumed by another program.

3 Methods

Problem Setting. Let the dataset be $D = \{(x_i, t_i)\}_{i=1}^n$, where x_i is a math problem and $t_i \in \{\text{MCQ}, \text{FRQ}\}$ indicates its type. For public examples, gold answers y_i are available and can be used for evaluation; for private examples, only x_i and type information are available. The model produces a text response

$$r_i \sim p_\theta(\cdot \mid s(t_i), u(x_i)),$$

where $s(t_i)$ is a type-specific system prompt and $u(x_i)$ is a user prompt containing the problem and, for MCQ, the answer choices. The final submitted response is

$$\hat{r}_i = \begin{cases} h_{\text{mcq}}(r_i) & \text{if } t_i = \text{MCQ}, \\ h_{\text{frq}}(r_i, x_i) & \text{if } t_i = \text{FRQ}. \end{cases}$$

For MCQ, h_{mcq} is intentionally light and keeps the model response trace with a boxed letter. For FRQ, h_{frq} is a deterministic repair function that tries to produce a valid final boxed answer while preserving the raw response in the CSV. Accuracy is measured by the competition’s answer extractor and judge, not by BLEU or text similarity.

The public dataset contains 1126 total examples: 375 MCQ and 751 FRQ. The private set used for submission contains 943 examples: 300 MCQ and 643 FRQ. Because the private labels are hidden, model development relied on the public labels, random public subsets, and full public FRQ validation.

Submission Contract. The final submission contract was unusually important. Kaggle required exactly two CSV columns: `id` and `response`. The `response` field was not supposed to contain only the extracted answer; it was supposed to contain the full model response trace, including reasoning and a final boxed answer. Therefore, the pipeline could not simply submit a cleaned answer string. Instead, when repair was needed, the system appended or rewrote the final answer region while keeping the response as a plausible full trace. This was also important for academic-integrity verification: graders could re-run `run_inference()` and compare generated responses against the submitted CSV.

In code, the final pipeline enforces this contract with four checks before writing the CSV: every private `id` appears exactly once, no response is blank, every row has a final `\boxed{\}` expression, and MCQ final boxes contain valid option letters. These checks prevented several submission-format failures during the final hours of the project.

Idea Summary. The key idea is that the problem is partly a reasoning task and partly an answer-interface task. Early experiments showed that the base Qwen thinking model already solved many problems in its reasoning trace. A large fraction of errors came from output formatting: missing boxes, wrong answer count, overly verbose final answers, numbers with units, or correct intermediate values not selected as the last boxed expression. Instead of trying to fine-tune the model into a new solver, I focused on making the inference and extraction path more reliable.

The final system is deliberately conservative. MCQ performance was strong in the baseline setup, so I did not keep prompt changes that lowered MCQ accuracy. FRQ performance was improved through a stricter prompt and a repair layer. This approach has risks: deterministic repairs can overfit public templates, and template-specific rules may not transfer to unseen private questions. I mitigated this by prioritizing generic repairs first, using template solvers only when the problem text matched a clear mathematical pattern, and preserving the raw response trace so the grader could still extract an answer if the appended repair was ignored.

Method Description. The final pipeline has four stages, shown in Figure 1. First, it loads the private dataset and splits examples by whether they contain answer choices. Second, it runs MCQ inference with the original baseline prompt. Third, it runs FRQ inference with a concise prompt that includes answer-count and precision directives. Fourth, it repairs FRQ responses and merges MCQ and FRQ outputs into the required CSV.

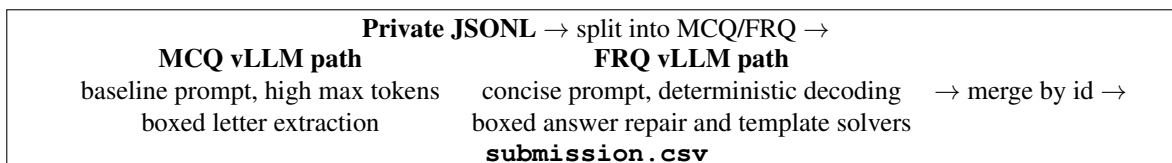


Figure 1: Final inference pipeline. MCQ and FRQ are run separately so that GPU memory is released between phases and so each type can use its own prompt and decoding settings.

For MCQ, the system prompt is: read the problem and choices, select the single best answer, and output only the letter inside `\boxed{\}`. Options are rendered as A, B, C, etc. This was kept close to the starter notebook because experiments showed that adding extra reasoning instructions hurt MCQ.

For FRQ, the system prompt tells the model to solve efficiently, write exactly one boxed answer, respect the number and order of `[ANS]` blanks, avoid units unless required, avoid thousands commas, prefer exact expressions, and obey rounding instructions. The user prompt also adds a gold-free structural directive: if a problem has k `[ANS]` blanks, output exactly k comma-separated values inside a single box. This targets the largest FRQ formatting failures without using labels.

The repair layer is implemented in `scripts/repair_frq_outputs.py`. It generates candidate final answers from multiple sources: existing post-processing, last boxed answer, explicit “answer” markers, right-hand sides after equals signs, labelled values, compact option letters, numeric-token extraction, SymPy simplification, tuple wrapping, and problem-template solvers. Candidates are ranked conservatively. A replacement is selected only when a rule is confident enough to prefer it over the original final box.

Repair Candidate Categories. The repair module uses generic rules before template-specific ones. Generic repairs include:

- **Box extraction:** select the last valid `\boxed{\}` expression, while avoiding boxes that contain unfinished prose.
- **Answer-marker extraction:** parse text after phrases such as “the answer is,” “final answer,” and “therefore.”
- **Expression normalization:** remove units when they are not required, strip thousands commas, normalize semicolons and vertical bars to comma-separated lists, and simplify expressions with SymPy when safe.
- **Answer-count repair:** compare the number of extracted values with the number of `[ANS]` blanks and reject candidates with obvious count mismatches.

Algorithm 1 Final inference pipeline

-
- 1: Load `data/private.jsonl`.
 - 2: Split rows into MCQ and FRQ examples.
 - 3: Load Qwen3-4B-Thinking with vLLM and bitsandbytes for MCQ.
 - 4: Generate MCQ responses with baseline MCQ prompt and high max-token budget.
 - 5: Release the MCQ engine and clear GPU memory.
 - 6: Load Qwen3-4B-Thinking with vLLM and bitsandbytes for FRQ.
 - 7: Generate FRQ responses in batches of five with deterministic decoding.
 - 8: For each FRQ response, build repair candidates from boxes, answer markers, numeric expressions, labels, and matched templates.
 - 9: Select a final answer candidate only if it passes confidence checks.
 - 10: Merge MCQ and repaired FRQ responses by `id`.
 - 11: Validate row count, uniqueness, non-empty responses, and boxed answers.
 - 12: Write `submission.csv`.
-

- **MCQ-letter recovery:** for embedded choice blanks, convert compact answer strings such as ACABC into A, C, A, B, C.

Template repairs are only applied when the problem text matches a recognizable pattern. Examples include half-life equations, tangent general solutions, sample standard deviation tables, chi-square tests, confidence intervals, binary addition, simple geometric formulas, percent-change word problems, trigonometric feature extraction, and repeated algebra templates. These rules were responsible for many of the later public-set gains, but they were also the highest-overfit component. For that reason, the final report treats them as deterministic engineering rather than evidence that the model itself learned more mathematics.

4 Experiments

4.1 Baselines

I compared several baselines and alternatives:

- **Starter vLLM baseline.** Qwen3-4B-Thinking with vLLM, bitsandbytes quantization, and starter-style prompts.
- **Modified MCQ prompts.** Variants that added more detailed reasoning instructions. These were rejected because they reduced MCQ performance.
- **Concise FRQ prompting.** A prompt that forces answer count, one boxed answer, and exact formatting.
- **Longer-token FRQ generation.** FRQ max tokens increased from 4096 to 6144. This was rejected because the model rambled more and public FRQ accuracy fell.
- **LoRA/SFT adapters.** Public self-distillation and external math data adapters. These were rejected because held-out or sampled FRQ accuracy did not reliably beat the repaired base system.
- **Deterministic repair system.** The final selected method, combining base vLLM generation with generic and template-specific answer repair.

4.2 Evaluation

The only metric optimized was answer accuracy under the competition judge. For public FRQ experiments, I used `Judge.auto_judge()` and reported the fraction of examples judged correct. For Kaggle submissions, I used the public leaderboard score. I did not optimize text similarity, explanation quality, or latency except as practical constraints. This choice matches the competition: the grader extracts final answers from the submitted response trace.

4.3 Implementation details

All final inference ran on RunPod with a single NVIDIA A40 GPU with 48 GB VRAM and CUDA 12.8. The final full private run took approximately 10–12 hours. The repository now exposes one entry point:

```
run_inference(data_path=data/private.jsonl; output_csv=submission.csv).
```

The function runs the complete pipeline and writes the required CSV. The final notebook simply imports and calls this function.

During development I used terminal-first execution for long jobs. Most runs were launched with `nohub` and logged to `results/logs/*.log` so that inference could continue after disconnecting from Jupyter Lab. This mattered because full FRQ or private inference could take many hours, and browser sessions occasionally disconnected even when the GPU job was still alive. Short jobs such as rescoring or summarizing were run directly in the terminal. The final repository does not require the exploratory logs, but this workflow was essential for producing repeatable experiments under time pressure.

Table 1: Final inference hyperparameters. MCQ keeps the original high-token baseline behavior, while FRQ uses deterministic decoding and a shorter context to reduce rambling.

Setting	MCQ	FRQ
Model	Qwen/Qwen3-4B-Thinking-2507	
Serving	vLLM, bitsandbytes quantization/load format	
Temperature	0.6	0.0
Top-p	0.95	1.0
Top-k	20	-1
Max output tokens	32768	4096
Max model length	16384	8192
Batch size	all prompts or configured batch	5
GPU memory utilization	0.50	0.78

The optimizer settings for LoRA experiments were not part of the final submission, but they informed the decision to avoid fine-tuning. I tried QLoRA/LoRA with rank 8–16, alpha 16–32, dropout 0.05, learning rates around $3 \cdot 10^{-5}$ to $5 \cdot 10^{-5}$, batch size 1, gradient accumulation 16, and maximum length 4096. Public-only self-distillation improved formatting on a small holdout after repair but did not provide a robust enough gain. External OpenR1/math-style training was mismatched with the competition’s short final-answer format and was also rejected.

4.4 Results

Result 1: public FRQ repair substantially improved answer accuracy. Table 3 summarizes the public FRQ development. The most important result is that deterministic repair improved the full public FRQ set from 430/751 (57.26%) to 544/751 (72.44%). This gain came from format recovery and template-specific solving rather than changing the base model.

The error analysis after the final repair showed 425 raw-correct examples, 119 examples corrected after post-processing, 109 remaining expression-format errors, 64 wrong-math errors, 28 wrong-answer-count errors, and 6 precision/rounding errors. This breakdown supports the main design decision: many remaining gains were available from interface repair, but the largest unsolved category after repair was genuine mathematical error.

The remaining 207 public FRQ errors after final repair were not all equally fixable. The `wrong_math` cases usually required a new reasoning path or a symbolic solver. The `wrong_answer_count` cases often came from multi-blank problems where the model solved only part of the prompt or output an extra intermediate value. The `expression_format` cases were mixed: some were truly equivalent expressions that the public judge did not accept, while others were long prose fragments mistakenly extracted from the final answer. The repair oracle ceiling of

Table 2: Development timeline. These stages explain why the final system is an inference-and-repair system rather than a fine-tuned model.

Stage	What happened
Environment setup	vLLM 0.8.5.post1 with bitsandbytes was installed on RunPod. Later package pinning fixed tokenizer/version issues and avoided CUDA driver conflicts.
Baseline restoration	MCQ accuracy dropped when prompts were changed. The MCQ path was restored to the starter-style vLLM configuration.
FRQ full baseline	A full public FRQ run showed that many errors were format-related, not purely mathematical. This motivated answer repair.
Post-processing	Generic extraction and normalization recovered answers hidden in model traces, improving public FRQ from the high 50s to low 60s.
Template repair	Repeated public error groups were converted into deterministic solvers, eventually reaching 544/751 on public FRQ.
Fine-tuning attempts	Public self-distillation and external math LoRA adapters were tested but failed held-out checks or degraded behavior. They were excluded from the final submission.
Final packaging	The codebase was cleaned into one notebook, <code>run_inference.py</code> , inference scripts, repair scripts, requirements, and README.

Table 3: Public FRQ development trajectory. The final selected public FRQ repair version reached 544/751, with an oracle ceiling of 546/751 for the available candidate set.

System variant	Public FRQ correct	Accuracy
Base FRQ generation / early repair baseline	430/751	57.26%
First stronger repair pass	463/751	61.65%
Template repair v3–v5	515/751	68.58%
Final template repair v6/v10	544/751	72.44%
Available repair oracle ceiling	546/751	72.70%

546/751 showed that, for the existing generated traces, only two more public examples were recoverable by candidate selection alone. Further gains would have required new model generations or new deterministic solvers, not merely better ranking.

Result 2: leaderboard gains came from preserving MCQ and improving FRQ. Table 4 shows the staged public leaderboard improvements. The final public score was 0.674. A key empirical finding was that MCQ should be left mostly unchanged. When MCQ prompts were modified, performance dropped; when MCQ was restored to the original vLLM baseline, the system became more stable. FRQ improvements then drove the remaining gains.

Table 4: Kaggle public leaderboard progression. Scores are public leaderboard scores observed during the final submission process.

Submission stage	Public score
Initial full-response submission	0.650
Targeted safe version	0.653
Cleaned last-shot submission	0.667
Improved final-answer repair submission	0.671
Final aggressive-but-safe submission	0.674

The gap between public FRQ repair accuracy (72.44%) and Kaggle public score (0.674) is expected because the leaderboard combines MCQ and FRQ, evaluates on a hidden split, and may contain a different distribution from public

FRQ validation. The final system was still the strongest observed submission.

Result 3: fine-tuning did not beat the repaired base pipeline. LoRA was attractive because it promised to improve FRQ without adding many hand-written rules. In practice it was not reliable. The public self-distillation adapter trained on base-correct public FRQ examples reached 95/200 raw accuracy and 110/200 after repair on a fixed public holdout, which was not enough to replace the base pipeline. A larger external OpenR1/math-data adapter generated faster and shorter answers, but its training targets did not match the competition’s exact final-answer style. The result was worse answer behavior on sampled FRQ. Because the final requirement was reproducible private inference, I rejected all adapters and used the base model only.

Result 4: longer output budgets were harmful. The 6144-token FRQ run tested whether the thinking model simply needed more space. It did not. On the full public FRQ set, 6144 tokens produced 419/751 raw accuracy and 422/751 after post-processing, below the 4096-token path. Longer outputs often contained the correct answer early, but then continued into contradictory analysis. This result directly shaped the final hyperparameters: FRQ stayed at 4096 max output tokens, while MCQ retained the high-token baseline because MCQ outputs were shorter and more stable.

4.5 Ablations

Table 5: Ablation-style findings. Not every experiment was run under perfectly identical conditions because of time and GPU constraints, but each influenced the final design.

Change	Observation	Decision
Modify MCQ prompts	More detailed MCQ reasoning instructions reduced MCQ performance relative to the baseline path.	Rejected
FRQ structural prompt	Counting [ANS] blanks and forcing one boxed answer reduced wrong-answer-count and formatting failures.	Kept
Increase FRQ max tokens to 6144	Public full FRQ generation fell to 422/751 after post-processing, worse than the 4096-token repaired baseline.	Rejected
Public self-distill LoRA	A 200-example holdout reached 110/200 after repair, not enough to justify replacing the base model.	Rejected
External math LoRA	External solution-style data mismatched the desired concise answer format and degraded sampled FRQ behavior.	Rejected
Deterministic repair layer	Improved full public FRQ from 430/751 to 544/751.	Kept

The main ablation lesson is that more model freedom was not always better. Longer generation and extra reasoning often increased contradictions. Fine-tuning was also not automatically helpful. The strongest gains came from constraining the output interface and using deterministic recovery for cases where the model’s trace already contained enough information.

5 Discussion

Achievements. The project produced a complete, reproducible inference system that can be run from a notebook or from a terminal command. The public FRQ pipeline improved by 15.18 percentage points over the 430/751 baseline, reaching 544/751. The final Kaggle public score reached 0.674. The final repository is much cleaner than the exploratory codebase: it contains one entry point, one notebook, three inference/repair scripts, dependency pins, and a README explaining setup and runtime.

The most important achievement is not only the final score, but also the conversion from an exploratory notebook into a reproducible grader-facing pipeline. The final code no longer depends on manual copying of intermediate JSONL

files, private-label access, or one-off terminal repair commands. Calling `run_inference()` loads the model, runs both question types, repairs FRQ responses, validates the CSV, and writes the final file. This directly satisfies the competition's final-code requirement.

Bottlenecks and Mitigation. The first bottleneck was GPU memory. vLLM with bitsandbytes solved most memory issues, but MCQ and FRQ had to be run separately to avoid out-of-memory failures and to free KV cache between phases. The second bottleneck was generation time. A full private run took most of a day, so I used random 200-example FRQ samples and public full-set rescoring to iterate quickly. The third bottleneck was model overthinking. The Qwen thinking model often continued debating after reaching the answer. I mitigated this with a concise prompt and by reducing FRQ max tokens back to 4096. The fourth bottleneck was format brittleness. I addressed it with a repair module that extracted and normalized answer candidates from the model trace.

A second-order bottleneck was project management under a fixed deadline. It was tempting to keep launching larger runs, but each full private run cost close to a submission cycle. To control this, I used a two-level validation pattern: quick sampled public runs for speed, then full public FRQ repair before trusting a change. This pattern rejected several ideas that looked plausible from a small sample but were not robust enough for final submission.

Overfitting Risk. The final repair system has some overfitting risk because template solvers were derived from public-set error analysis. I tried to reduce this risk in three ways. First, the final pipeline never uses private labels; all private predictions are generated from private problem text only. Second, generic repairs are preferred over narrow templates whenever possible. Third, template rules are text-pattern-based mathematical solvers, not direct memorized public IDs. Still, because the private set appears to come from the same problem distribution, these templates are likely helpful but not guaranteed to transfer perfectly. This is why the report distinguishes public FRQ repair accuracy from true private generalization.

Lessons Learned. The biggest lesson is that leaderboard performance for reasoning models depends heavily on the boundary between language generation and evaluation. A mathematically correct paragraph can still fail if the last boxed answer is malformed. Another lesson is that fine-tuning is not automatically beneficial under a short project timeline. The LoRA experiments consumed useful time and sometimes made the model worse because the training targets were mismatched with the competition's exact output requirements. A final lesson is that preserving a strong baseline matters. I initially tried to improve both MCQ and FRQ, but the best path was to keep MCQ unchanged and focus engineering effort on FRQ.

With more time, I would build a cleaner validation protocol with a held-out public split before doing any template-specific repair. I would also train a smaller answer-extraction model or verifier that chooses among candidate final answers from the trace, rather than relying on hand-ranked deterministic rules. Finally, I would implement symbolic solvers for broader classes of algebra, statistics, and geometry templates so that the repair layer generalizes beyond the public set.

What I Would Do Next. If the project continued, the next step would not be another broad LoRA run. The best next step would be a verifier-and-solver hybrid. The verifier would take the model trace, the problem, and multiple candidate answers, then choose the most likely final answer. The solver component would handle high-frequency algebra/statistics templates deterministically. This would target the remaining errors more directly than fine-tuning, because the final public errors were split between wrong math, wrong answer count, and expression formatting. A second possible direction is controlled self-consistency: generate a small number of short FRQ answers with different seeds, then select only when candidates agree. My attempted multi-generation run was too broad and expensive; a focused version on high-uncertainty FRQ categories could be more useful.

Reproducibility Checklist. The final repository contains:

- `README.md`, documenting the GPU type, model, setup, runtime, and entry point.

- `requirements.txt`, pinning the core inference dependencies.
- `run_inference.py`, exposing the required `run_inference()` function.
- `starter_code_cse151b_comp.ipynb`, a notebook wrapper for graders.
- `scripts/eval_mcq_vllm.py`, `scripts/eval_frq_vllm.py`, and `scripts/repair_frq_outputs.py`, containing the runnable inference and repair code.

No final LoRA weights are required because the selected system uses the public Hugging Face base model `Qwen/Qwen3-4B-Thinking`.

Team Responsibilities. Keith Gong handled the project end-to-end: environment setup on RunPod, vLLM migration, prompt design, public FRQ evaluation, error analysis, repair module engineering, LoRA/SFT experiments, Kaggle submission packaging, README cleanup, and final notebook integration. The final code exposes the required `run_inference()` function and is organized so that graders can reproduce the submission from the notebook or command line.

References

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020. doi:[10.48550/arXiv.2005.14165](https://arxiv.org/abs/2005.14165). URL <https://arxiv.org/abs/2005.14165>.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. doi:[10.48550/arXiv.2106.09685](https://arxiv.org/abs/2106.09685). URL <https://arxiv.org/abs/2106.09685>.
- [3] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023. doi:[10.48550/arXiv.2309.06180](https://arxiv.org/abs/2309.06180). URL <https://arxiv.org/abs/2309.06180>.
- [4] Qwen Team. Qwen3-4B-Thinking-2507 model card. <https://huggingface.co/Qwen/Qwen3-4B-Thinking-2507>, 2025. Hugging Face model card, accessed 2026-06-01.
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. doi:[10.48550/arXiv.1706.03762](https://arxiv.org/abs/1706.03762). URL <https://arxiv.org/abs/1706.03762>.
- [6] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. doi:[10.48550/arXiv.2203.11171](https://arxiv.org/abs/2203.11171). URL <https://arxiv.org/abs/2203.11171>.
- [7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. doi:[10.48550/arXiv.2201.11903](https://arxiv.org/abs/2201.11903). URL <https://arxiv.org/abs/2201.11903>.
- [8] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li,

Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. doi:[10.48550/arXiv.2505.09388](https://doi.org/10.48550/arXiv.2505.09388). URL <https://arxiv.org/abs/2505.09388>.